

Project Report: Placement Portal

Name: Subhadeep Dey

Roll Number: 24dp2000029

1. Introduction and Objective

1.1 Problem Statement

In many educational institutions, the placement process is handled manually or through fragmented systems (like emails and spreadsheets). This leads to inefficiencies such as duplicate applications, miscommunication between companies and the placement cell, and a lack of real-time status tracking for students.

1.2 Objective

The objective of this project is to build a centralized, responsive, and secure web application—the Placement Portal. This platform bridges the gap between the Institute's Placement Cell (Admin), recruiting organizations (Companies), and the candidates (Students). The portal streamlines the entire recruitment lifecycle: from company registration and job posting to student applications and final status tracking, all within a secure, Role-Based Access Control (RBAC) environment.

2. Technologies Used

The application is built using a modern, decoupled tech stack focusing on scalability, security, and a seamless user experience.

2.1 Backend Framework & Libraries

- Python 3: The core programming language.
- Flask: A lightweight WSGI web application framework used for routing, request handling, and backend logic.
- Flask-SQLAlchemy: An ORM (Object Relational Mapper) used to define Python classes that map to the database tables, allowing for complex queries without writing raw SQL.
- Flask-Login: Utilized for secure user session management and authentication handling.
- Werkzeug: Used for robust password hashing (`generate_password_hash`, `check_password_hash`) and secure file uploading (`secure_filename`).

2.2 Frontend Stack

- HTML5 & CSS3: For page structure and advanced styling (e.g., Glassmorphism effects).
- Bootstrap 5: A CSS framework used strictly via CDN for a highly responsive grid layout and interactive components (Modals, Cards, Badges, Accordions).
- Jinja2: The templating engine used to dynamically render HTML based on backend Python data.
- Chart.js: A JavaScript library integrated to render dynamic, interactive data visualizations (e.g., Application Status Doughnut Chart).

2.3 Database

- SQLite: A lightweight, disk-based database used for persistent data storage during the development and production lifecycle.

3. Database Schema Design

The database is highly relational, utilizing Primary Keys (PK) and Foreign Keys (FK) to link users, profiles, drives, and applications.

3.1 User Table (Authentication & Roles)

The foundational table handling authentication for all roles.

- id (Integer, PK)
- username (String, Unique, Not Null)
- password (String, Not Null) - Stores hashed passwords.
- role (String, Not Null) - Defines the RBAC scope ('admin', 'company', 'student').

3.2 StudentProfile Table

Stores student-specific metadata. Has a one-to-one relationship with User.

- id (Integer, PK)
- user_id (Integer, FK -> User.id)
- full_name (String, Not Null)
- department (String, Nullable)
- skills (String, Nullable)
- resume_file (String, Nullable) - Stores the generated unique filename of the uploaded PDF.
- is_blacklisted (Boolean, Default: False)

3.3 CompanyProfile Table

Stores organizational data. Has a one-to-one relationship with User.

- id (Integer, PK)
- user_id (Integer, FK -> User.id)
- company_name (String, Unique, Not Null)
- is_approved (Boolean, Default: False) - Requires Admin intervention.
- is_blacklisted (Boolean, Default: False)

3.4 PlacementDrive Table

Represents a job posting. Has a many-to-one relationship with CompanyProfile.

- id (Integer, PK)
- company_id (Integer, FK -> CompanyProfile.id)
- job_title (String, Not Null)
- job_description (Text, Not Null)
- eligibility_criteria (Text, Not Null)
- application_deadline (DateTime, Not Null)
- status (String, Default: 'Pending') - Tracks if the drive is Pending, Approved, or Closed.

3.5 Application Table

A mapping table handling the many-to-many relationship between Students and Placement Drives.

- id (Integer, PK)
- student_id (Integer, FK -> StudentProfile.id)
- drive_id (Integer, FK -> PlacementDrive.id)
- application_date (DateTime, Default: Current UTC)
- status (String, Default: 'Applied') - Tracks the candidate pipeline (Applied, Shortlisted, Selected, Rejected).

4. System Architecture & Features

The application adheres to the MVC (Model-View-Controller) pattern, ensuring a clean separation of concerns.

4.1 Security and Access Control

- Role-Based Access Control (RBAC): Every route is protected by @login_required and explicit role checks (if current_user.role != '...'). This ensures a student cannot access the company dashboard, and unauthorized users cannot hit API endpoints.
- Password Hashing: Passwords are never stored in plain text.

4.2 Module 1: Admin Management

The Admin acts as the gatekeeper of the portal.

- **Dashboard Statistics:** Displays aggregate counts of Students, Companies, Drives, and Applications, alongside a dynamic Chart.js doughnut graph visualizing overall application statuses.
- **Entity Search:** Integrated GET-based search bars to filter companies by name, and students by name or ID.
- **Company & Drive Approvals:** Admins must explicitly toggle approvals for new companies to ensure legitimacy. They also govern the status of placement drives.
- **Blacklisting:** Admins can blacklist rogue companies or non-compliant students, instantly revoking their system privileges.
- **Data Previews:** Admins can click on entities to trigger responsive Modals, revealing detailed Student Profiles (including PDF resumes) and Drive Specifications without leaving the page.

4.3 Module 2: Company / Recruiter Portal

- **Drive Creation:** Approved companies can create job postings specifying titles, detailed descriptions, eligibility, and deadlines.
- **Application Review:** Companies can view a list of applicants for their specific drives, cross-referencing their skills and departments.
- **Resume Access:** Direct links to safely open student-uploaded PDF resumes in a new tab.
- **Status Tracking:** Recruiters can update an applicant's lifecycle status seamlessly via dropdown forms (Applied -> Shortlisted -> Selected/Rejected).

4.4 Module 3: Student Experience

- **Profile Management:** Students can update their major/department, list core skills, and upload a PDF resume. The system uses `secure_filename` and appends unique IDs to prevent file collisions.
- **Job Discovery:** Students view a tailored list of 'Approved' drives. The query logic strictly filters out drives the student has already applied to, preventing duplicate application errors.
- **Application History:** A persistent table tracks every job the student has applied to and reflects real-time status updates made by the recruiting companies.

5. API Design & Integration (Optional Enhancements)

To demonstrate decoupled architecture and readiness for mobile app integration, several RESTful JSON API endpoints were implemented. These APIs are strictly secured using RBAC to prevent data leaks.

1. GET /api/drives
 - Access: All authenticated users.
 - Description: Returns a structured JSON list of all currently active and approved placement drives.
2. GET /api/companies
 - Access: All authenticated users.
 - Description: Returns a JSON list of all approved organizations.
3. GET /api/students
 - Access: Restricted to Admins and Companies only.
 - Description: Returns a directory of student names, departments, and skills. (Excludes sensitive IDs and authentication data).
4. GET /api/applications/<drive_id>
 - Access: Restricted to Admins and the specific Company that owns the drive_id.
 - Description: Returns statistical summaries and an array of applicant data and statuses for a specific job posting.

6. UI / UX Design Implementation

- Glassmorphism Aesthetic: The portal avoids monotonous flat design by implementing a global "Glassmorphism" CSS theme. A soft, blurred background image is overlaid with translucent white .card elements (backdrop-filter: blur), creating a modern, premium feel.
- Single Page Application (SPA) Feel: Extensive use of Bootstrap 5 Modals was implemented. Instead of redirecting users to separate pages for "Drive Details" or "Student Profiles", detailed data loads instantly in centered pop-ups, significantly reducing page load times and improving UX.
- Responsiveness: Native Bootstrap grid classes (col-md-8, col-md-4, row) guarantee that the portal automatically scales and stacks perfectly on mobile devices, tablets, and desktop monitors without requiring external CSS frameworks.

7. Installation and Execution

1. Environment Setup: Ensure Python 3.x is installed.
2. Dependencies: Install required libraries using:
`pip install -r requirements.txt`
(Requires Flask, Flask-SQLAlchemy, Flask-Login, Werkzeug)
3. Database Initialization: The application uses Flask app contexts to automatically generate the placement_portal.db file and the default admin user on the first run.
4. Run Server: Execute the application via the terminal:
`python app.py`

5. Access: Open a web browser and navigate to <http://127.0.0.1:5000/>.

8. Conclusion

The Placement Portal successfully implements a highly structured, role-based ecosystem using Flask. By fulfilling all core requirements—including complex database relationships, secure file handling, and dynamic filtering—and implementing advanced enhancements like JSON APIs and Chart.js integrations, the portal proves to be a robust, scalable, and production-ready solution for campus recruitment management.

9. Acknowledgements and AI Assistance

During the development of this Placement Portal, Artificial Intelligence (AI) tools were utilized as an educational pair-programming assistant to enhance productivity, code quality, and the overall design process. Specifically, AI was leveraged for:

- **Framework Suggestions:** Recommending best practices for the Flask ecosystem, including the use of Flask-Login for secure RBAC implementation and Werkzeug for secure file uploading (secure_filename).
- **Debugging Support:** Assisting in identifying and resolving logic errors, such as fixing complex SQLAlchemy relationship queries and troubleshooting CSS conflicts that caused Bootstrap Modals to freeze the screen layout.
- **Styling and UI/UX:** Providing guidance and vanilla CSS snippets to achieve the modern "Glassmorphism" design aesthetic, ensuring a cohesive visual theme across all dashboards without violating the restriction of using external CSS frameworks other than Bootstrap.

10. Video Presentation Link: https://drive.google.com/file/d/19SfoFFW_AIxKDHJFM-jXS0qkMmndQPM_/view?usp=sharing